

Aula 007

. Quiz de abertura

São seis trechos curtos (Python, Java, JS, C) para a turma palpitar a saída. Eles cobrem precedência, concatenação, coerção, comparações encadeadas e o `or` do Python. As respostas aparecem nos slides seguintes:

1. `-4 512`
2. `3345`
3. `3 52`
4. `0`
5. `padrão True`
6. `true false true`

. Expressões aritméticas: questões de projeto

Quem projeta uma linguagem precisa definir:

- a precedência e a associatividade dos operadores;
- a ordem de avaliação dos operandos;
- se efeitos colaterais são permitidos;
- se o usuário pode sobrecarregar operadores;
- que mistura de tipos é aceita.

O slide mostra que `a + b * c` (com `a=3`, `b=4`, `c=5`) dá 35, 23 ou 27 conforme a regra adotada. APL avalia sempre da direita para a esquerda. Lisp dispensa a precedência porque usa notação prefixa com parênteses.

. Precedência e associatividade na prática

Uma tabela compara Python, JS e Java:

- `-2 ** 2` dá -4 em Python e erro de sintaxe em JS. Java não tem `**`.
- `**` é associativo à direita: `2 ** 3 ** 2 = 512`.
- `1 + 2 + "3"` dá `TypeError` em Python e `"33"` em JS e Java.
- `-7 / 2` e `-7 % 2` dão resultados diferentes em cada linguagem.

Os operadores `*`, `/`, `%`, `+`, `-` associam à esquerda, enquanto `**`, `++` e `--` associam à direita. Há duas exceções curiosas: o `^` do Visual Basic associa à esquerda, e no Excel `=-2^2` dá 4. A recomendação final é usar parênteses na dúvida.

. Ordem dos operandos e efeitos colaterais

No exemplo do livro, a função `fun1()` altera `"a"` e o programa calcula `a = a + fun1()`. O resultado é 8 em Java, onde a ordem é garantida da esquerda para a

direita. Em C (gcc) e em Go a ordem não é definida e o resultado foi 20. Sem efeitos colaterais há transparência referencial: a ordem deixa de importar, e $f(x) + f(x)$ pode virar $2 * f(x)$.

. Operadores sobrecarregados

Um mesmo símbolo pode ter vários significados. Em Java, + soma e concatena. Em C++, & é tanto o "E bit a bit" quanto o "endereço de", o que é um risco. C++, C#, Python, Rust e Kotlin permitem sobrecarga definida pelo usuário, mas Java não permite (por isso BigDecimal usa .add()). O exemplo em Python é uma classe Vetor com __add__ e __mul__. Em Java, 'a' + 1 dá 98, mas "" + 'a' + 1 dá "a1".

. Conversões de tipos: alargamento, estreitamento e coerção

- Um alargamento pode perder precisão: um int grande convertido em float muda de valor.
- O estreitamento exige cast explícito, por exemplo (byte) 200 = -56.
- Em Java, byte + byte vira int.
- $1 / 2 = 0$, enquanto $1 / 2.0 = 0.5$.

Uma tabela mostra quais linguagens convertem int para real implicitamente. C, Java e Python convertem; Go, Rust e ML/F# não, e dão erro de compilação ao misturar tipos. A lição é que a coerção esconde erros.

. Coerção implícita: JavaScript e PHP

JS tem resultados estranhos como "5" - 2 = 3, "5" + 2 = "52" e [] + {} = "[object Object]". O == do JS não é transitivo: 0 == "" e 0 == "0" são verdadeiros, mas "" == "0" é falso. O PHP tem o "type juggling": "0e123" == "0e456" é true, porque as duas strings são lidas como notação científica e valem 0. Por isso dois hashes MD5 diferentes podem ser considerados iguais. A regra prática é usar === e !==.

. Expressões relacionais e booleanas

Em C, o booleano é um inteiro. Assim, 3 > 2 > 1 vira 1 > 1, que dá 0. Python trata comparações encadeadas como (a > b) and (b > c). O slide lista os valores que contam como falsos em Python. C, Python e JS aceitam qualquer valor no if, enquanto Java, C#, Go e Rust exigem um booleano.

. Avaliação em curto-circuito

Em Java, && faz curto-circuito e & não, o que pode causar NullPointerException. Em JS, ?. acessa propriedades com segurança. Já || troca o 0 por 10, enquanto ?? só substitui null/undefined. É preciso cuidado com efeitos colaterais: em (a > b)

`|| (b++ / 3)`, o `b++` pode não executar. Em Python, `and` e `or` devolvem um dos operandos, não necessariamente um booleano.

. Sentenças de atribuição

Em C, C++ e JS a atribuição é uma expressão. Isso permite `sum = count = 0` e `while ((ch = getchar()) != EOF)`, mas também faz `if (x = y)` compilar por engano. Nas outras linguagens:

- Java e C# também tratam a atribuição como expressão, mas o `if` exige booleano.
- Python só tem atribuição como expressão no operador morsa `:=`.
- Em Go, nem `x++` é expressão.
- Em Rust, a atribuição vale `()`.

Python também permite atribuição múltipla, como `a, b = b, a`. Em linguagens funcionais, `val/let` dão nome a um valor imutável.

. Atribuição de modo misto

Java só permite alargamento implícito. Mesmo assim, `i += 3.7` compila porque esconde um cast. C converte em qualquer direção: `int i = 3.7` vira 3, e `unsigned u = -1` vira 4294967295. A armadilha clássica é `double taxa = acertos / total * 100` dar 0.0, porque a divisão inteira acontece antes do alargamento.

. Estruturas de controle: quantas são necessárias?

Böhm e Jacopini (1966) mostraram que sequência, seleção e laço lógico bastam para qualquer algoritmo, e o `goto` não é necessário. Estruturas extras como `for`, `switch` e `foreach` existem pela facilidade de escrita. O limite é a legibilidade: quanto maior a linguagem, menos você reconhece do código alheio.

. Seleção de dois caminhos: o "else pendente"

No exemplo em C, a indentação sugere que o `else` pertence ao primeiro `if`. Na verdade, em Java, C, C++ e C#, o `else` pertence ao `if` mais próximo que ainda não tem `else`, e a indentação é ignorada (exceto em Python e F#). Linguagens com chaves obrigatórias, como Perl, Go e Rust, eliminam o problema. Rust e F# tratam o `if` como expressão, então ele pode devolver um valor (`let y = if x > 0 { x } else { 2 * x }`). Python tem a expressão condicional `y = x if x > 0 else 2 * x`.

. Seleção múltipla: do switch de C ao match

- **C:** sem break, o controle "cai" para os casos seguintes. Com `index = 1`, executa odd, even e o default.
- **Java 14+:** o switch com `->` é uma expressão, não cai entre casos e é exaustivo para enums.
- **Rust:** o match com faixas precisa ser exaustivo, e sem o `_` não compila.
- **Go:** cada caso termina sozinho, e cair para o próximo exige `fallthrough`.
- **C#:** cair implicitamente é proibido.
- **Python 3.10:** tem match com padrões e guardas.

. Laços controlados por contador

Em C, o for é um laço lógico disfarçado. Alterar `i` dentro do corpo muda a contagem (imprime 0 3 6 9), e todas as expressões do cabeçalho são opcionais. Em Python, o for percorre um iterável, então `i += 2` dentro do corpo não muda nada (imprime 0 a 9). Já `range(10, 0, -3)` dá 10 7 4 1. As questões de projeto são:

- qual é o tipo e o escopo da variável de controle;
- se ela pode ser alterada no corpo;
- se os parâmetros são avaliados uma vez ou a cada volta.

. Laços lógicos, saídas e iteradores

- **Java:** o break rotulado (`break externo`) sai dos dois laços aninhados de uma vez.
- **JavaScript:** `for...in` percorre as chaves, que são strings, e por isso a soma vira "0012". Já `for...of` percorre os valores e dá 60.
- **Bug do Zune (2008):** no último dia de um ano bissexto (`dias == 366`), nada muda dentro do laço, que fica infinito.
- **Laço pós-teste (do-while):** executa ao menos uma vez, e isso é fácil de esquecer.

. Desvio incondicional: o goto

O caso real é o "goto fail" da Apple (2014). Uma linha `goto fail;` duplicada desviava sempre, então a verificação da assinatura nunca era alcançada. Como `err` valia 0, o certificado era aceito como válido. Chaves obrigatórias ou o `gcc -Wall` (aviso de endentação enganosa) teriam pegado o bug. Dijkstra (1968) criticou o goto em "Go To Statement Considered Harmful". Hoje C, C++, C# e Go têm goto, mas Java não. `break`, `continue` e `return` funcionam como "gotos restritos" e melhoram a legibilidade.

. Comandos protegidos de Dijkstra

A sintaxe é `if x >= y -> max := x [] y >= x -> max := y fi`.

- Todas as guardas são avaliadas.

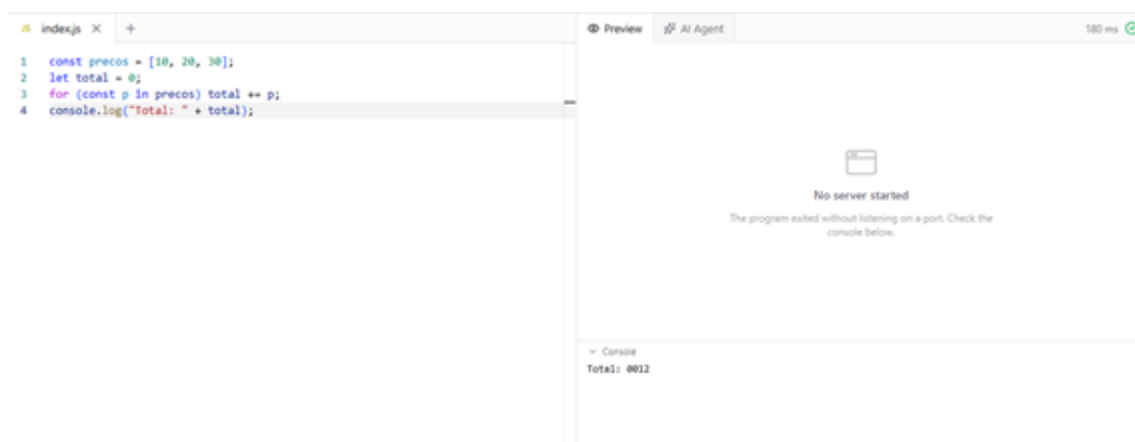
- Se várias são verdadeiras, a escolha é não determinística.
- Se nenhuma é verdadeira, dá erro (no `if`) ou termina o laço (no `do . . . od`).

Essa ideia é a base do CSP de Hoare (1978) e do `select` de Go. Quando os dois canais estão prontos, o Go sorteia um deles: em 1000 repetições, o resultado fica perto de 505 contra 495 e muda a cada execução.

. Exercício em dupla: encontre o bug (1/3)

1) JavaScript

a) Saída:



```

1 const precos = [10, 20, 30];
2 let total = 0;
3 for (const p in precos) total += p;
4 console.log("Total: " + total);

```

Preview AI Agent 180 ms

No server started
The program exited without listening on a port. Check the console below.

Console
Total: 0012

- b) O bug está na soma, pois esta gerando o resultado errado e com zero na frente.

2) Python

a) Saída:



```

1 def desconto(preco, d=None):
2     d = d or 10 # padrão: 10%
3     return preco * (100 - d) / 100
4
5 print(desconto(200, 0))

```

main.py X + Console I/O AI Agent 136 ms

180.0

- b) O bug está no resultado que está mostrando o preço final, porém, o correto seria mostrar o quanto de desconto foi dado.

. Exercício em dupla: encontre o bug (2/3)

3) Java

a) Saída:

```
J Main.java × +
1 int dia = 2; String nome = "";
2 switch (dia) {
3     case 1: nome = "domingo";
4     case 2: nome = "segunda";
5     case 3: nome = "terça";
6     default: nome = "inválido";
7 }
8 System.out.println(nome);
9
10
```

Console I/O AI Agent Exit 1

Main.java:2: error: statements not expected outside of methods and initializers
switch (dia) {
^

Main.java:8: error: class, interface, annotation type, enum, record, method or field expected
System.out.println(nome);
^

2 errors

Debug this error

b) O que causa os bugs é a falta de métodos.

c) Colocar o métodos.

4) C

a) Saída:

```
C Main.c × +
1 int saldo = 100, saque = 50;
2 if (saque > saldo);
3 | printf("saldo insuficiente\n");
4 printf("fim\n");
5
6
```

Console I/O AI Agent Exit 1

Main.c:2:1: error: expected identifier or '(' before 'if'
2 | if (saque > saldo);
| ^~

Main.c:3:12: error: expected declaration specifiers or '...' before string constant
3 | printf("saldo insuficiente\n");
| ^~~~~~

Main.c:4:8: error: expected declaration specifiers or '...' before string constant
4 | printf("fim\n");
| ^~~~~~

Debug this error

b) Falta de declaração.

c) Colocar declaração.

5) Go

a) Saída:

```
main.go ×
1 x := 10
2 if x > 5 {
3     x := x * 2
4     fmt.Println("dentro:", x)
5 }
6 fmt.Println("fora:", x)
7
```

Console I/O AI Agent Exit 1

main.go:1:1: expected 'package', found x

Debug this error

b) Falta package.

c) Colocar package.

6) Java

a) Saída:

J Mainjava × +

```
1 int acertos = 7, total = 10;  
2 double taxa = acertos / total * 100;  
3 System.out.println(taxa + "%");  
4  
5
```

Console I/O AI Agent Exit 1

Main.java:3: error: class, interface, annotation type, enum,
record, method or field expected
System.out.println(taxa + "%");
^
1 error
[Debug this error](#)

- b) Falta método.
- c) Colocar método.